Detecting Logical Bugs in Built-in SQL Functions via Function Property–Driven Metamorphic Testing

Abstract

Built-in SQL functions are a critical component of modern database management systems (DBMSs), providing essential operations for data transformation and computation across a wide range of queries. However, the complexity of these functions makes them prone to bugs, particularly logical bugs that can silently corrupt query results without causing explicit failures. These logical bugs are difficult to detect as they often produce plausible yet incorrect outputs, undermining the integrity of data-driven applications.

In this paper, we propose *Function Property-Driven Metamorphic Testing* (FPMT), a novel approach to detecting logical bugs in built-in SQL functions. Our key insight is that many SQL functions satisfy well-defined semantic properties (e.g., oddness, idempotence, and inverse relations) that can be leveraged to construct property-equivalent queries. By embedding these functions into complex, context-rich SQL queries and applying function-specific metamorphic relations, FPMT generates mutated queries and compares their outputs to detect inconsistencies, indicating logical bugs. We evaluate FPMT across six widely used DBMSs—MySQL, MariaDB, OceanBase, Percona, PolarDB, and TiDB—and demonstrate its effectiveness in uncovering previously unknown logical bugs in built-in SQL functions. In total, FPMT identified 44 logical bugs that were missed by existing approaches, among which 42 have been confirmed by DBMS developers.

1 Introduction

Built-in SQL functions are a foundational component in modern database management systems (DBMSs). They provide pre-defined operators for data transformation and computation, and are extensively used throughout SQL queries—from projections and predicates to grouping/aggregation and nested expressions [23, 36]. In practice, DBMSs expose a large and continuously evolving function space, covering arithmetic/mathematical functions, string functions, date/-time functions, type-conversion and conditional functions, as well as aggregate and analytic functions [23, 36]. This diversity is essential for expressive data processing, yet it also means that each function family must satisfy its own domain constraints, type-specific corner cases, and semantic rules [23, 36]. As a result, implementing and optimizing built-in SQL functions correctly is inherently complex.

Due to this substantial complexity, built-in SQL functions are prone to bugs during development and long-term maintenance [10]. Their implementations often intertwine intricate control logic with heterogeneous functionality and type-specific corner cases. A particularly subtle yet serious class of defects is **logical bugs** in built-in SQL functions, which are notably troublesome for three reasons.

- **(1) Broad impact**. Built-in SQL functions are repeatedly exercised across diverse queries and workloads; thus, a bug in a widely used function can be triggered at scale and silently propagate incorrect intermediate values through downstream operators [10].
- **(2) High severity**. Even without crashing the DBMS, logical bugs may corrupt query results and undermine the integrity of data-driven applications; in security-sensitive settings, malformed inputs to function implementations may further escalate into denial-of-service [35].
- **(3) Hard to expose**. Detecting logical bugs in built-in SQL functions is challenging because effective testing must (i) embed built-in SQL functions into complex, context-rich SQL queries where they interact with surrounding operators (e.g., predicates, joins, and subqueries) under rich typing and semantic rules, and (ii) provide reliable test oracles, as logical bugs typically cause the DBMS to return plausible but incorrect results rather than exhibiting apparent failures (e.g., crashes), allowing them to easily go unnoticed in conventional testing [11, 12, 14].

Figure 1 shows a real logical bug in Percona (PS-10441 [25]). The original query groups tuples by `FLOOR(ASIN(hpg31.c1))` and computes an aggregate output `MIN(hpg31.c5)`. The mutated query modifies the `GROUP BY` clause by adding `FLOOR(-ASIN(-hpg31.c1))`. Semantically, this mutation should be equivalent: for any numeric input `x`, `ASIN(x) = -ASIN(-x)`, and therefore `FLOOR(ASIN(x))` should equal `FLOOR(-ASIN(-x))`. As a result, adding the second grouping key should not change the set of groups, nor the aggregated value `MIN(hpg31.c5)`; the two queries are expected to return identical outputs. However, the DBMS produces inconsistent results: the mutated query returns a different `col_1` value compared to the original query, while no crash or runtime error is raised. This discrepancy indicates a logical bug—the system returns a incorrect result that violates the expected query-level consistency relation under an aggregation context where the built-in function expression is used as a grouping key.

Recent work SoFT [10] aims to uncover bugs in built-in SQL functions. SoFT starts by mining DBMS documentation and regression test suites to compile a list of built-in functions, and then extracts concrete function expressions from existing SQL statements in the test suite. It synthesizes new test cases by mutating the arguments of these extracted functions according to boundary-value generation patterns, and

```
CREATE TABLE t1 (c1 INT ,c5 DATE ,c6 VARCHAR(10) );
CREATE TABLE t3 (c1 INT ,c4 YEAR ,c5 TIME ;
INSERT INTO t1 (c1, c5, c6) VALUES (9505, '2025-12-02', 'sample_h');
INSERT INTO t3 (c1, c4, c5) VALUES (4992,2020, '2023-01-01 22:44:35');

-- Original Query
(
  SELECT
        DISTINCT MIN(hpg31.c5) AS col_1,
        GROUP_CONCAT(hpg31.c4) AS col_2,
        FLOOR(ASIN(hpg31.c1)) AS col_3
  FROM t3 AS hpg31
  GROUP BY FLOOR(ASIN(hpg31.c1)), FLOOR(ASIN(hpg31.c1))
)
UNION ALL
SELECT
        nhp53.c5 AS col_1, nhp53.c6 AS col_2, nhp53.c1 AS col_3
FROM t1 AS nhp53;
--result:   2026-02-03 22:44:35 |  2020 |  NULL  --(expected)
--result:   0000-00-00 00:00:00 |  2020 |  NULL  --(actual)

-- Mutated Query
(
  SELECT
        DISTINCT MIN(hpg31.c5) AS col_1,
        GROUP_CONCAT(hpg31.c4) AS col_2,
        FLOOR(ASIN(hpg31.c1)) AS col_3
  FROM t3 AS hpg31
  GROUP BY FLOOR(-ASIN(-hpg31.c1)), FLOOR(ASIN(hpg31.c1))
)
UNION ALL
SELECT
        nhp53.c5 AS col_1, nhp53.c6 AS col_2, nhp53.c1 AS col_3
FROM t1 AS nhp53;
--result:   2026-02-03 22:44:35 |  2020 |  NULL
```

Figure 1. Queries exposing a logical bug in a built-in SQL function in Percona Server.

executes the resulting statements to detect failures, using crash signals as the oracle [10]. While effective for exposing crash bugs caused by improper boundary handling, SoFT has two limitations with respect to our goal. First, since it primarily operates at the argument-mutation level within existing statements without modeling function semantics (e.g., argument/return types and constraints) or their interactions with surrounding query contexts, it is difficult to systematically embed built-in functions into complex, context-rich SQL where subtle interactions often arise. Second, it is insufficient for logical bugs: the DBMS may return plausible but incorrect results without any visible failures, which calls for semantic test oracles beyond crash detection.

There have also been several studies on detecting logical bugs in DBMSs [6, 11, 14, 29–34]. For instance, TLP [30] partitions queries based on predicate logic and NoREC [29] rewrites queries to disable DBMS optimizations. These techniques are effective at uncovering planner/optimizer issues (e.g., incorrect rewrites, join logic, or predicate evaluation) by enforcing query-level consistency constraints. However, their primary focus is on validating query semantics at the relational-operator and plan/optimization level, rather than the correctness of built-in SQL function implementations during expression evaluation. In contrast, our work focuses on a different class of correctness issues: data-operation-level logical bugs rooted in built-in SQL function implementations. Such bugs occur during function expression evaluation and may manifest only when functions are embedded in non-trivial query contexts.

In this paper, we propose *Function Property–Driven Metamorphic Testing* to systematically detect logical bugs in built-in SQL functions. Our key insight is that many built-in SQL functions satisfy checkable semantic properties (e.g., oddness, idempotence, and inverse relations), and these properties naturally serve as invariants to build semantic test oracles for logical-bug detection. Specifically, FPMT embeds target functions into context-rich queries and constructs a mutated query by rewriting a function expression into a property-equivalent one while keeping the surrounding query context unchanged. The oracle checks whether the original and the property-equivalent mutated queries produce identical outputs; any discrepancy indicates that the DBMS violates the function property and thus exposes a logical bug. Because the transformation is confined to the function expression, we can generate an equivalent mutated query without analyzing or constraining the full query structure, and the same oracle remains valid even when the function is evaluated under non-trivial contexts such as joins, aggregations, and nested subqueries. For example, in Figure 1, we applies a an oddness-preserving rewrite to `ASIN` to produce an equivalent mutated query; the observed output discrepancy directly indicates a violation of the function-property invariant and exposes a logical bug.

Based on this insight, we propose a novel and general approach, ***F**unction **P**roperty–Driven **M**etamorphic **T**esting* (FPMT), which can effectively find logical bugs in built-in SQL functions. FPMT starts by constructing a function model for each built-in SQL function, capturing its signature, constraints, and properties. Using these models, FPMT generates an original query where the target functions are effectively embedded into a rich-context SQL query. Then, based on predefined transformation rules, FPMT creates a mutated query by applying property-equivalent transformations to the function expressions while keeping the surrounding context unchanged. Finally, FPMT compares the results of the original and mutated queries; any discrepancy in the outputs indicates a logical bug in the SQL function implementation. The key intuition behind the effectiveness of this approach is that the transformed function expression forces the DBMS to use different code logic when executing the modified query. This cross-validation between different query executions is expected to produce identical results, thus revealing and exposing any logical bugs.

We implement our approach as a practical DBMS testing tool and evaluate it on six widely-used and extensively-tested DBMSs: MySQL, MariaDB, OceanBase, Percona, PolarDB, and TiDB. In total, FPMT found 44 logical bugs in built-in SQL functions, including 8 in MySQL, 7 in MariaDB, 5 in OceanBase, 10 in Percona, 9 in PolarDB, and 5 in TiDB.

Among these bugs, 42 have been confirmed. These results show the effectiveness of FPMT in uncovering logical bugs in DBMSs and highlight its potential to identify previously unknown SQL function bugs in widely tested systems.

Overall, we make the following contributions:

- We propose a novel methodology *Function Property–Driven Metamorphic Testing* for systematically detecting logical bugs in built-in SQL functions.
- We implement FPMT as a DBMS testing tool, which automatically generates, mutates, and verifies SQL queries based on *Function Property–Driven Metamorphic Testing*, enabling the detection of logical bugs in SQL functions.
- We evaluate FPMT on six real-world DBMSs and find 44 previously unknown logical bugs. To further facilitate research on DBMS testing, we open-source FPMT at https://anonymous.4open.science/status/FPMT-DBTesting-D3CA.

2 Background and Motivation

2.1 Background

DBMSs and SQL Functions. A database management system (DBMS) is a software system that enables users to define, create, maintain databases, and control access to stored data [39]. Structured Query Language (SQL) is the standard language for interacting with relational DBMSs, supporting data definition, manipulation, and querying [38]. Within SQL, SQL functions are language constructs that take one or more inputs and return a value, allowing users to perform in-query computation, data transformation, formatting, and other data-processing tasks. SQL functions can be broadly categorized into two types.

- **Built-in functions** [23, 36]: functions natively integrated into the DBMS, predefined by the vendor to support common operations. Their implementations and corner-case behaviors may vary across DBMSs. Figure 2 shows a representative dialect difference for string search: MySQL uses `LOCATE(substr, str)`, while PostgreSQL uses `POSITION (substr IN str)` to implement a similar operation. A DBMS processes built-in SQL functions by resolving input arguments, applying type conversions, and invoking the function logic to return a value.
- **User-defined functions (UDFs)** [22, 37]: functions created by users to implement application-specific logic beyond what built-in functions provide. UDFs extend the DBMS with customizable computation to meet specialized processing needs.

Unless otherwise stated, we use "SQL functions" to refer to "built-in SQL functions" throughout this paper. Unlike functions in general-purpose programming languages that are invoked along explicit control flow and may depend on mutable program state, SQL functions are evaluated declaratively as part of query execution. Their results are consumed by surrounding relational operators, and their inputs may also be

```sql
-- MySQL Function LOCATE(substr, str)
SELECT id, username, LOCATE('Bob', username) AS pos FROM users
WHERE LOCATE('Bob', username) > 0;

-- PostgreSQL Function POSITION(substr IN str)
SELECT id, username, POSITION('Bob' IN username) AS pos FROM users
WHERE POSITION('Bob' IN username) > 0;
```

Figure 2. Dialect-Specific String Search Functions in SQL.

shaped by the query context. The MySQL query in Figure 2 illustrates the tight coupling between SQL functions and their surrounding query context. The output of `LOCATE('Bob', username)` is immediately consumed by the `WHERE` predicate to filter tuples. Meanwhile, its input `username` is not a fixed constant but a per-tuple value provided by the query execution context, so the function is evaluated over context-dependent inputs rather than as an isolated call. Accordingly, this paper treats SQL functions as black-box components and tests them through their observable query behaviors in realistic SQL contexts.

SQL Function Bugs. When DBMS processes SQL function expressions, bugs in function implementations (or in the surrounding process pipeline) can lead to incorrect behaviors. In general, a bug can be triggered by SQL statements that contain carefully crafted function expressions, potentially resulting in severe consequences such as DBMS crashes, denial-of-service or even arbitrary code execution in extreme cases [10].

Detecting SQL function bugs is particularly critical. First, built-in SQL functions are core building blocks of database operations and are extensively used for processing diverse data types. For example, MySQL provides 90 built-in functions for string manipulation and computation [23]. As a result, once a frequently used built-in function is defective, the bug can be repeatedly triggered across a large number of SQL statements and silently propagate erroneous intermediate values through multiple execution stages and downstream operators, potentially causing cascading failures and interoperability issues in dependent applications [10]. Second, built-in SQL function bugs can pose significant security and reliability threats. Beyond memory-safety flaws that may lead to DBMS crashes, denial-of-service, or even arbitrary code execution in extreme cases [35], logical bugs can also be security-relevant in practice: when buggy function semantics are used inside predicates, constraints, or security-critical checks, the DBMS may return plausible but incorrect results that effectively bypass filtering or integrity assumptions, undermining the correctness and trustworthiness of data-driven applications.

From a testing perspective, SQL function bugs can be broadly categorized into two types. *Crash bugs* manifest as explicit failures during query execution, such as DBMS crashes, assertion failures, or runtime errors, and are therefore detectable using failure-based oracles. In contrast, *logical bugs*

do not necessarily trigger any visible failure; instead, they cause the DBMS to return incorrect results, silently violating the intended semantics of built-in functions. Consequently, detecting logical bugs faces the classical test oracle problem: determining the correct output for comparison. This oracle challenge is further amplified for built-in SQL functions, whose correctness depends on rich typing rules, implicit casts, `NULL` semantics, and subtle interactions with surrounding query operators. In this work, we focus on logical bugs in built-in SQL functions, as they are harder to expose than crash bugs and remain under-explored in prior testing.

2.2 Challenges

Despite their importance, logical bugs in built-in SQL functions remain difficult to expose in practice. We identify two key challenges:

- **(C1) Context-rich embedding.** Many function bugs do not manifest in isolated calls (e.g., `SELECT f(x)`), but only when functions are embedded in complex SQL and interact with surrounding operators such as joins, aggregations, windows, and (correlated) subqueries under rich typing and semantic rules. For example, in Figure 1, the problematic behavior is exposed only when the function expression is used as a `GROUP BY` key and its groups drive an aggregate `MIN`; a standalone evaluation of `SELECT FLOOR(ASIN(·))` is insufficient to surface the inconsistency.
- **(C2) Oracle design for logical bugs.** Function logical bugs often produce plausible but incorrect results without visible failures (e.g., crashes), which makes it necessary to develop reliable semantic test oracles rather than relying on crash-based signals.

2.3 Limitations of Existing Approaches

To detect logical bugs, existing techniques typically generate SQL queries to test DBMSs and then check whether the returned results satisfy expected correctness conditions. The two mainstream lines are differential testing and metamorphic testing.

(1) Differential testing compares results across multiple DBMS implementations to identify inconsistencies. However, it is not effective when queries rely on dialect-specific features or DBMS-specific semantics—because built-in functions whose definitions and behaviors may vary across systems. For example, as shown in Figure 2, MySQL and PostgreSQL use different functions for string searching, which makes cross-DBMS comparisons difficult [11].

(2) Metamorphic testing is the most popular technique for detecting logical bugs in DBMSs. It generates a mutated query from an original query such that their results should satisfy a predefined relation, referred to as a metamorphic relation (MR). For example, NoREC [29] rewrites an optimizable query into a non-optimizable form and detects semantic inconsistencies by comparing their outputs, while TLP [30] partitions query predicates into multiple subqueries and verifies that the union of their results is equivalent to the original query result. These approaches are effective at uncovering planner/optimizer issues by enforcing query-level consistency constraints. However, they are not well-suited for logical bugs rooted in built-in SQL functions. First, existing methods operate at a coarse query-granularity, limiting their ability to embed functions into diverse execution contexts. To preserve metamorphic relations, these approaches must reason about query semantics, yet SQL's flexibility and rich operator interactions make complete semantic understanding intractable. Consequently, they restrict generated queries to simplified patterns that avoid complex function interactions, failing to construct the context-rich settings where function bugs manifest (Challenge C1). Second, existing MRs are designed around relational operators and query structure, and can suffer from *oracle pollution* when the same buggy function evaluation is exercised in both the original and mutated queries, allowing the checks to pass even though both outputs are incorrect. Their oracles are not *function-semantic* aware: they do not encode function-level semantic identities as the expected relation (Challenge C2).

For example, in the Percona bug PS-10441 [25] shown in Figure 1, mainstream metamorphic oracles such as NoREC and TLP are difficult to apply. NoREC relies on rewriting an optimizable query into a semantically equivalent non-optimizable form, but the original query already contains complex grouping and set operations (e.g., `GROUP BY` with function keys and `UNION ALL`), leaving limited room for a meaningful "disable-optimization" rewrite while preserving equivalence. TLP partitions predicates in `WHERE`/`HAVING` and checks union-equivalence, yet in this example the critical semantics are encoded through a function-based grouping key inside a subquery, whose effect on the final output is implicit and context-dependent. Partitioning predicates inside such a subquery does not guarantee that the union of the partitioned results remains equivalent to the original result, because the grouping/aggregation behavior couples tuples across partitions in a non-trivial way. More importantly, even if these oracles can run, they may still fail to expose the bug due to oracle pollution: both the original and mutated queries exercise the same buggy evaluation of `FLOOR(ASIN(·))`, so the oracle can observe "consistent" yet jointly incorrect outputs and thus cannot distinguish the logical bug.

2.4 Key Insights

To detect logical bugs in built-in SQL functions in a general and systematic way, we propose a new methodology—*Function Property-driven Metamorphic Testing (FPMT)*, which shifts the focus from preserving the semantics of a whole query to preserving well-defined semantic properties of built-in function expressions. Specifically, many built-in functions satisfy mathematical or semantic identities (e.g., oddness,

idempotence, and inverse relations). We treat these properties as MRs and validate a DBMS by checking whether query outputs respect the expected property-induced relation, without requiring a ground-truth oracle for the entire query. This methodology directly addresses the two challenges of function bug detection.

Solving (C1): context-rich embedding via function modeling. FPMT starts from a function model that captures each built-in function's signature, constraints and properties. This modeling is essential because SQL functions are not evaluated in isolation: their outputs are consumed by surrounding relational operators (e.g., predicates, grouping keys, join conditions), and their inputs are shaped by the query context (e.g., per-tuple column values, derived expressions). With the function model, FPMT can (i) generate *type-correct* and *domain-valid* function expressions, (ii) place them into query positions with compatible expected types, and (iii) compose nested functions while preserving well-typedness and respecting constraints. Therefore, FPMT can embed built-in functions into complex SQL queries without introducing semantic errors, so that the generated tests execute successfully and expose logical bugs under non-trivial query contexts.

Solving (C2): function-semantic oracles that generalize across query structure. On top of this modeling, FPMT defines metamorphic relations directly from *function properties* and uses them as function-semantic test oracles. Crucially, these oracles do not rely on understanding or constraining the global query structure: since the expected relation is defined at the function level, it remains valid even when the function is embedded in complex SQL with joins, aggregations, or subqueries. This yields an oracle that is both *function-semantic* and *context-robust*, allowing FPMT to operate in the context-rich settings required by (C1).

2.5 Motivating Example

Figure 1 illustrates how FPMT detects the Percona bug PS-10441. FPMT first embeds the target function `ASIN` into a context-rich query by using it as a `GROUP BY` key alongside aggregation and set operations, ensuring the function is evaluated under interactions with surrounding relational operators. It then generates a follow-up query by applying the odd-symmetry property `ASIN(x) = -ASIN(-x)` to mutate the function expression while keeping the surrounding query structure unchanged. By comparing the outputs of the original and mutated queries, FPMT detects a discrepancy, thereby exposing a logical bug.

3 Function Property-driven Metamorphic Testing

In this section, we propose *Function Property-driven Metamorphic Testing* (FPMT) to systematically detect logical bugs in built-in SQL functions. FPMT leverages function properties

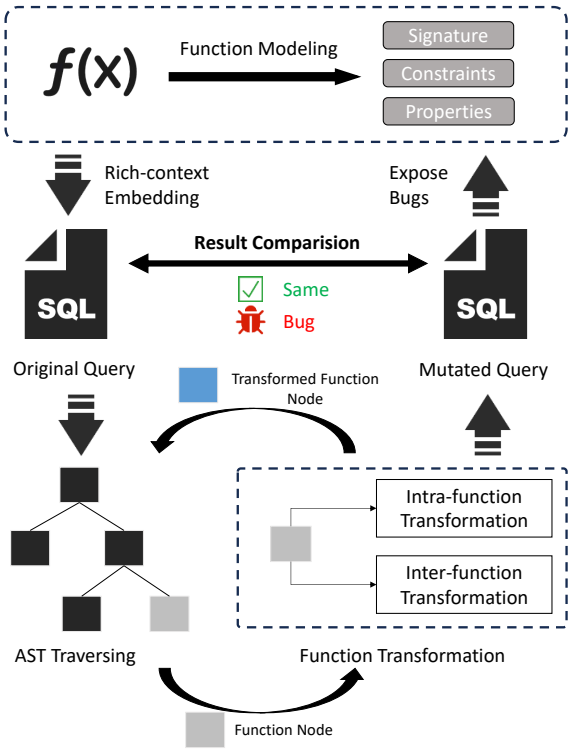

Figure 3. Overview of FPMT.

to construct property-equivalent mutated queries and uses result consistency as the oracle, without requiring ground-truth query outputs. We will present the formulation and algorithm of FPMT.

3.1 Overview

Figure 3 illustrates the end-to-end workflow of FPMT. FPMT targets a large set of built-in SQL functions and validates their behaviors in SQL query contexts where they interact with surrounding operators. It first performs function modeling to build a model for each function, summarizing its signature, constraints, and properties (Section 3.2). Guided by these models, FPMT generates function expressions, embeds them into rich-context SQL, and synthesizes an original query (Section 3.3). It then traverses the query abstract syntax tree (AST) to locate function nodes and applies function transformation to construct a mutated query, using either intra-function transformation or inter-function transformation (Section 3.4). Finally, FPMT executes the original and mutated queries and performs result comparison; any inconsistency between their outputs exposes bugs.

3.2 Function Modeling

FPMT constructs an explicit *function model* for built-in SQL functions to support (i) type-correct generation and context-aware embedding of function expressions and (ii) property-driven transformations with checkable semantic oracles. For

each built-in function f, the model consists of three parts: **Signature**, **Constraints**, and **Properties**. We formalize them below and provide concrete examples.

Definition 3.1 (Function Model)**.** For each built-in SQL function f, FPMT builds a model

$$\mathcal{M}(f) = \langle \mathsf{Sig}(f),\ \mathsf{Con}(f),\ \mathsf{Prop}(f) \rangle,$$

where $\mathsf{Sig}(f)$ captures the callable interface of f, $\mathsf{Con}(f)$ records semantic constraints required for valid execution, and $\mathsf{Prop}(f)$ specifies function properties used to instantiate metamorphic transformations.

3.2.1 Signature.

Definition 3.2 (Signature)**.** A function signature is represented as

$$\sigma = \langle name,\ kind,\ k,\ (\tau_1, \dots, \tau_k),\ \tau_r \rangle,$$

where *name* is the function identifier, *kind* is the function category such as aggregation function, k is the arity, $(\tau_1, \dots, \tau_k)$ are the parameter type requirements, and τ_r is the return type. A built-in function may have multiple overloads; we denote its overload set as $\mathsf{Sig}(f) = \{\sigma_1, \sigma_2, \dots\}$. For variadic functions, FPMT models them as a finite family of overloads by instantiating k within a bounded range.

This signature-based modeling serves two purposes. First, it captures the typing constraints of function calls in a DBMS as explicit, checkable specifications (parameter types and return type). Second, it unifies overloaded and variadic functions under the same representation, enabling FPMT to enumerate valid call forms by selecting a signature from $\mathsf{Sig}(f)$.

Example 3.1 (Overloads of `CONCAT`)**.** `CONCAT` can be modeled as a family of scalar overloads with different arities. For example, two overloads are

$$\langle \texttt{CONCAT},\ \text{scalar},\ 1,\ (\text{String}),\ \text{String} \rangle$$

$$\text{and} \quad \langle \texttt{CONCAT},\ \text{scalar},\ 3,\ (\text{String}, \text{String}, \text{String}),\ \text{String} \rangle.$$

This allows FPMT to instantiate `CONCAT` with different numbers of string arguments while keeping the call type-correct.

3.2.2 Constraints.

Many functions impose semantic restrictions beyond type correctness (e.g., valid input ranges or restrictions on special arguments). We model such requirements as predicates over argument expressions.

Definition 3.3 (Constraints)**.** For a function f with a selected overload σ of arity k, let its argument expressions be $\vec{e} = (e_1, \dots, e_k)$, where e_i denotes the i-th argument expression. FPMT associates σ with a (possibly empty) list of per-argument constraints

$$\mathsf{Con}(f, \sigma) = \langle c_1, \dots, c_k \rangle,$$

where each c_i is either $\emptyset$ (no extra restriction) or a predicate over e_i. A function call $f(\vec{e})$ is *admissible* if every non-empty constraint is satisfied:

$$\bigwedge_{i=1}^{k} \big(c_i = \emptyset \ \vee\ c_i(e_i) \big) \ \text{ holds.}$$

In our model, constraints capture semantic preconditions required for valid evaluation beyond type correctness. For a selected overload σ, a function call is considered admissible only if its argument expressions satisfy all predicates in $\mathsf{Con}(f, \sigma)$.

Example 3.2 (Constraint of `DATE_FORMAT`)**.** For the scalar overload $\sigma = \langle$`DATE_FORMAT`, scalar, 2, (DateTime, String), String$\rangle$, FPMT models its constraints as a per-argument list

$$\mathsf{Con}(\texttt{DATE_FORMAT}, \sigma) = \langle \emptyset,\ c_{fmt} \rangle,$$

where the first argument has no extra restriction and the second argument must satisfy a format-template constraint:

$$c_{fmt}(f): \quad f \in \mathcal{G}_{fmt}.$$

Here, $\mathcal{G}_{fmt}$ denotes the grammar of valid format templates, i.e., strings composed of literal characters and supported format specifiers such as `%Y`, `%m`, `%d`, `%H`, `%i`, and `%s`. This constraint states that the second parameter of `DATE_FORMAT` must be a well-formed template string rather than an arbitrary string literal.

3.2.3 Properties.

Function properties capture semantic identities of SQL functions and serve as the specification basis for constructing property-equivalent expressions in later stages. A property is often tied to a specific overload and may constrain only a subset of its arguments (e.g., symmetry with respect to one argument, or commutativity between two arguments). Moreover, some properties relate multiple functions (e.g., inverse), which cannot be expressed by a single-function specification. We therefore model properties as *guarded equivalence templates* at the expression level.

Definition 3.4 (Property Template)**.** A property is represented as a guarded equivalence template

$$p = \langle \Sigma,\ I,\ \phi(\vec{x}),\ E(\vec{x}) \equiv E'(\vec{x}) \rangle,$$

where Σ is a set of one or more overload signatures involved in the property, I specifies the argument positions constrained by the property for each involved signature, $\phi(\vec{x})$ is the applicability precondition, and $E(\vec{x})$ and $E'(\vec{x})$ are two property-equivalent expressions under ϕ.

We categorize property templates into two types. *Intra-function properties* involve a single overload signature $\Sigma = \{\sigma\}$ and describe equivalences within the same function (e.g., idempotence). *Inter-function properties* involve multiple overload signatures $\Sigma = \{\sigma_1, \sigma_2, \dots\}$ and describe cross-function equivalences (e.g., inverse). For a function f, we denote the set of its property templates as $\mathsf{Prop}(f)$, which may include templates that reference other functions for inter-function cases.

Example 3.3 (Odd-symmetry property of ASIN)**.** For the scalar overload $\sigma = \langle \texttt{ASIN}, \text{scalar}, 1, (\text{Numeric}), \text{Numeric} \rangle$, an odd-symmetry property is modeled as an intra-function template with $I = \{1\}$:

$$\langle \{\sigma\}, \{1\}, (-1 \le x \le 1), \texttt{ASIN}(x) \equiv -\texttt{ASIN}(-x) \rangle.$$

Example 3.4 (Inverse property between EXP and LOG)**.** Let $\sigma_{\text{exp}} = \langle \texttt{EXP}, \text{scalar}, 1, (\text{Numeric}), \text{Numeric} \rangle$ and $\sigma_{\log} = \langle \texttt{LOG}, \text{scalar}, 1, (\text{Numeric}), \text{Numeric} \rangle$. An inverse/cancellation property is modeled as an inter-function template:

$$\langle \{\sigma_{\text{exp}}, \sigma_{\log}\}, \{1\}, (x > 0), \texttt{EXP}(\texttt{LOG}(x)) \equiv x \rangle.$$

This property modeling only specifies when two expressions are semantically equivalent under a precondition. How FPMT instantiates these templates to construct concrete transformations is presented in Section 3.4.

3.3 Embedding in Rich-context Query

This subsection presents how FPMT embeds a set of target functions into executable, rich-context SQL queries. Given a function set $\mathcal{F}$ to be tested, FPMT repeatedly synthesizes original queries by selecting a context-rich query skeleton and filling its expression slots with admissible function expressions. Each embedded call is instantiated from the function model $\mathcal{M}(f) = \langle \text{Sig}(f), \text{Con}(f), \text{Prop}(f) \rangle$: $\text{Sig}(f)$ ensures type correctness (allowed kind, arity, parameter/return types), while $\text{Con}(f, \sigma)$ provides admissibility predicates that are injected into the query to guarantee semantically valid evaluation.

Algorithm 1 synthesizes an executable original query by embedding a target built-in function into a context-rich SQL skeleton under the guidance of function models. It first selects and instantiates a query skeleton $\mathcal{T}$ with an expression slot s, and derives the slot contract $\langle \tau^\star(s), \mathcal{K}^\star(s) \rangle$ from the surrounding clause to determine the expected return type and allowed function kinds for s (Lines 1-4). It then samples a target function f from $\mathcal{F}$, chooses a compatible overload signature $\sigma \in \text{Sig}(f)$ whose kind and return type satisfy the slot contract, and instantiates the call arguments according to the parameter types $(\tau_1, \ldots, \tau_k)$ (Lines 5-9). For each argument position i, PickExpr constructs a candidate pool Ω consisting of (i) a well-typed column in the visible scope, (ii) a typed literal, (iii) a derived expression, and (iv) a nested function call that returns type τ_i , then samples an expression e_i from Ω (Lines 10-16). Importantly, PickExpr checks constraints during generation: for the i-th argument, it keeps re-sampling a candidate expression until the corresponding constraint c_i is satisfied (or no constraint is defined, i.e., $c_i = \emptyset$). After all arguments are instantiated, the algorithm forms the function expression $E = f(\vec{e})$, fills slot s with E, and outputs the finalized query Q (Lines 17-18). Due to space limitations, we provide a detailed example in our GitHub repository to facilitate understanding [1].

Algorithm 1 Embedding target functions into rich-context SQL queries

Require: Schema $\mathcal{S}$, target functions $\mathcal{F}$, models $\mathcal{M}(\cdot)$
Ensure: Executable query Q embedding at least one $f \in \mathcal{F}$
1: // **Step 1: Pick context**
2: Pick a skeleton $\mathcal{T}$ with an expression slot s.
3: Instantiate $\mathcal{T}$ with tables/aliases from $\mathcal{S}$.
4: Get slot contract $\langle \tau^\star(s), \mathcal{K}^\star(s) \rangle$.
5: // **Step 2: Pick function + overload**
6: Sample a non-empty set $\mathcal{F}_Q \subseteq \mathcal{F}$.
7: Sample $f \in \mathcal{F}_Q$ and assign it to slot s.
8: Pick $\sigma = \langle f, kind, k, (\tau_1, \ldots, \tau_k), \tau_r \rangle \in \text{Sig}(f)$.
9: Require $kind \in \mathcal{K}^\star(s)$ and $\tau_r \preceq \tau^\star(s)$.
10: // **Step 3: Build an admissible call**
11: Let $\text{Con}(f, \sigma) = \langle c_1, \ldots, c_k \rangle$.
12: $\vec{e} \leftarrow \langle \rangle$.
13: **for** $i \leftarrow 1$ to k **do**
14: $\quad e_i \leftarrow \textsc{PickExpr}(\tau_i, c_i, \mathcal{T}, \mathcal{F})$.
15: $\quad$ Append e_i to $\vec{e}$.
16: **end for**
17: $E \leftarrow f(\vec{e})$.
18: Fill slot s with E and output Q.
19: **return** Q.
20: **function** $\textsc{PickExpr}(\tau, c, \mathcal{T}, \mathcal{F})$
21: $\quad$ **repeat**
22: $\quad\quad \Omega \leftarrow \emptyset$.
23: $\quad\quad \Omega \leftarrow \Omega \cup \{\text{Column}(\tau, \mathcal{T})\}$ if such column exists.
24: $\quad\quad \Omega \leftarrow \Omega \cup \{\text{Literal}(\tau)\}$.
25: $\quad\quad \Omega \leftarrow \Omega \cup \{\text{Expr}(\tau, \mathcal{T})\}$.
26: $\quad\quad \Omega \leftarrow \Omega \cup \{\text{NestedFunc}(\tau, \mathcal{F}, \mathcal{T})\}$ if available.
27: $\quad\quad e \leftarrow \textsc{RandChoice}(\Omega)$.
28: $\quad$ **until** $c = \emptyset$ **or** $c(e)$ holds
29: $\quad$ **return** e.
30: **end function**

3.4 Property-driven Transformation

In this section, we describe how FPMT uses function properties to guide transformations and construct mutated queries. The main idea is to utilize the semantic properties of built-in SQL functions to generate property-equivalent transformations, which can then be used to detect logical bugs by comparing the results of original and mutated queries.

3.4.1 Transformation Rules. We categorize the transformation rules into *intra-function transformations* and *inter-function transformations* based on the nature of the function properties. These transformations are derived directly from the property templates defined in Section 3.2.3. The transformations apply within the scope of a single function expression or between multiple functions, depending on the property involved. For clarity, we denote func as an overload signature, and x as a parameter that exhibits the function property in relation to this signature. Table 1 illustrates the details of each transformation, including the applicable function expressions and the corresponding transformation rules. For example, intra-function transformations apply the property directly to a single function, such as transforming $\text{SIN}(x)$ into $-\text{SIN}(-x)$ based on the odd property. On

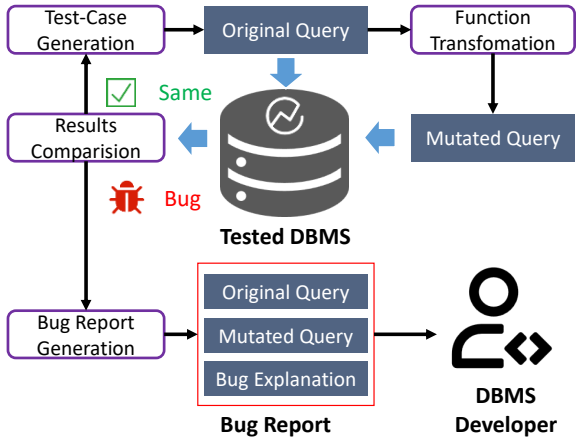

Figure 4. Implementation architecture of FPMT.

the other hand, inter-function transformations involve relationships between multiple functions, such as applying the invertible property between the exponential and logarithmic functions, where LN(EXP(x)) transforms into x. FPMT is designed to be extensible, allowing for the addition of new function properties and transformation rules as discussed in Section 4.2.

3.4.2 Mutation Process. The mutation process in FPMT systematically generates mutated queries by transforming function expressions based on their properties. It starts by parsing the original SQL query into its AST and identifying the relevant function expressions. Next, FPMT selects the appropriate function properties and applies predefined transformation rules to generate property-equivalent expressions. The transformed expressions are embedded back into the original query, replacing the function calls while leaving the rest of the query unchanged. This approach isolates the mutation to the function expressions, allowing for precise detection of logical bugs without interference from other query components.

4 Implementation

In this section, we discuss the implementation details of FPMT and the properties of this approach.

4.1 Implementation Details

We implemented FPMT into a fully automatic tool on top of ValScope [18], which we mainly use for generating databases and complex queries. The overall codebase of the tool is 17,970 lines of Python code, 2,980 of which is used to implement our approach. Figure 4 shows its architecture. The following describes the important implementation details.

Test-Case Generation. In the preparation phase, we begin by modeling each SQL function to be tested, as discussed in Section 3.2. We support six DBMSs, including 95 functions in MySQL, 92 functions in MariaDB, 86 functions in OceanBase, 95 functions in Percona, 91 functions in PolarDB and 76 functions in TiDB. These functions cover multiple categories, such as mathematical functions, string manipulation functions, date/time manipulation functions, type conversion functions, and conditional functions. All pre-built function models are stored in a function knowledge base.

Subsequently, these function models are used to guide the generation of original and mutated queries. During the original query generation phase, FPMT uses ValScope to generate rich-context queries with SQL slots. ValScope is an AST-based SQL query generator that provides a precise dependency analysis component. It achieves higher semantic accuracy than existing generators, such as SQLSmith and Go-randgen, particularly when handling complex queries with multiple nested subqueries. After generating these rich-context queries with SQL slots, we select functions from our function knowledge base to embed into the query, forming the original query. It is worth noting that this precise function modeling enables us to support complex query structures (especially nested functions) without introducing semantic errors.

Function Transformation. As described in Section 3.4, we first parse the original query using SQLGlot [19], which accepts the same context-free grammar used in seed query generation, to generate ASTs of the original queries for mutation. Next, we iterate over each function node, extract its overload, and match it with the pre-built models in our function knowledge base. We then randomly select transformation rules to apply mutations to the original query.

Result Comparison. FPMT compares the execution results of the mutated query and the original query, including their query outputs and the database changes triggered by these queries. Any discrepancies indicate the occurrence of a logical bug.

Bug Report Generation. To avoid repetitive bug reports and make the test results easier to understand, we first use SQLess [16] to simplify the bugs and pinpoint their root causes. Additionally, we follow the approach from Pinolo [11] to remove duplicates. The simplified original query, mutated query, and bug explanation will be submitted to the developers for confirmation.

4.2 Properties

Generality. FPMT demonstrates remarkable generality in testing built-in SQL functions across various DBMS testing tools, SQL dialects, and query structures. First, FPMT can be integrated into any DBMS testing tool, as long as the function model is constructed. Once the model for a target SQL function is established, it can be embedded into any DBMS testing framework for testing purposes. Second, FPMT can model SQL functions from different dialects. The function modeling process allows FPMT to represent functions from various SQL dialects based on their signature, constraints, and properties, making it adaptable to any DBMS dialect.

Table 1. Transformation Rules for Function Properties

Property	Transformation Rules	Example
	Intra-function Transformations	
Odd	`func(x) -> -func(-x)`	`SIN(x) -> -SIN(-x)`
Even	`func(x) -> func(-x)`	`COS(x) -> COS(-x)`
Idempotent	`func(x) -> func(func(x))`	`TRIM(str) -> TRIM(TRIM(str))`
Periodic	`func(x) -> func(x + T)`	`SIN(x) -> SIN(x + 2*PI())`
Commutative	`func(a, b) -> func(b, a)`	`ST_Equals(a, b) -> ST_Equals(b, a)`
	Inter-function Transformations	
Associative	`func(a, func(b, c)) -> func(func(a, b), c)`	`CONCAT(a, CONCAT(b, c)) -> CONCAT(CONCAT(a, b), c)`
Invertible	`inv(func(x)) -> x, func(inv(x)) -> x`	`LN(EXP(x)) -> x`

Note: The notation `func(x)` here does not mean the SQL function takes only one argument, but rather that the SQL function exhibits this property with respect to its argument `x`, aligning with the definition of properties.

Finally, FPMT also exhibits generality in its ability to handle different query structures. Since the test oracles are based on transformations at the function expression level rather than the global query structure, the approach remains effective across a wide variety of queries, regardless of their complexity or form.

Extensibility. FPMT is designed with extensibility in mind, enabling it to seamlessly adapt to different DBMS systems and support a wide range of SQL functions. Extending FPMT to a new DBMS requires minimal effort—only the implementation of the connection interface and function models for the target DBMS, which can typically be done in just 2 hours. In addition to the seven properties and their corresponding transformation rules mentioned in this paper, FPMT can be easily extended to support additional function properties and transformation rules. This can be achieved simply by adding new properties and their associated transformation rules to the function modeling, making it highly adaptable to new function behaviors.

Black-Box Testing. FPMT is a black-box technique that operates without access to DBMS internals, ensuring high portability across open-source and closed-source systems.

5 Evaluation

Our evaluation aims to answer the following questions to demonstrate the effectiveness of FPMT:

- ***Q1*** Can FPMT detect real logical bugs in SQL functions within widely used and extensively tested DBMSs? (Section 5.2)
- ***Q2*** How diverse are the logical bugs found by FPMT? (Section 5.3)
- ***Q3*** Can FPMT find logical bugs that are missed by existing approaches? (Section 5.4)

5.1 Experimental Setup

Tested DBMS. We evaluate FPMT on 6 widely used DBMSs, as summarized in Table 2. These DBMSs were chosen based on their popularity and extensive use in real-world applications. MySQL, MariaDB, TiDB, and OceanBase are widely used, with MySQL ranking second in the DB-Engines ranking, while MariaDB ranks 13th, and TiDB and OceanBase are also prominent players. Percona, a derivative of MySQL, and PolarDB, a cloud-native database, were selected to represent a broader spectrum of DBMSs. In our experiments, FPMT tests the latest versions of each DBMS. We run FPMT on each DBMS for 24 hours.

Table 2. The demographics of the DBMSs under test

DBMS	Version	GitHub Stars	DB-Engine	First Release
MySQL [20]	9.4.0	11.9K	2	1995
MariaDB [8]	12.0.2	6.7K	13	2009
OceanBase [21]	5.7.25	9.8K	99	2010
Percona [24]	8.0.43-34	1.2K	122	2008
PolarDB [5]	8.0.32	1.6K	105	2017
TiDB [27]	8.0.11	39.4K	85	2016

Environment. We run the experiments on one server with 104-cores Intel(R) Xeon(R) Gold 6230R CPU @2.10GHz and 500 GB memory. The server operates under the Ubuntu 20.04 OS, with the 5.4.0-135-generic version of the Linux kernel.

5.2 Bug Detection

Table 3 summarizes the status of the logical bugs found by FPMT in each DBMS. In total, we report 44 previously unknown logical bugs across six DBMSs, including 8 in MySQL, 7 in MariaDB, 5 in OceanBase, 10 in Percona, 9 in PolarDB, and 5 in TiDB. All of them are directly related to the built-in SQL function evaluation under non-trivial query contexts. Of these, 42 bugs have been confirmed by developers including 9 that have already been fixed, while the remaining 2 are still open and pending further verification.

Bug Importance. Although anecdotal, developer feedback is an important indicator of an approach's effectiveness and the significance of the bugs found. Two DBMS companies reached out to us regarding our testing efforts; both were interested in how we discovered the bugs, and one

Table 3. Status of logical bugs found by FPMT

DBMS	Reported	Confirmed	Fixed	Pending
MySQL	8	8	0	0
MariaDB	7	7	7	0
OceanBase	5	4	0	1
Percona	10	10	0	0
PolarDB	9	9	2	0
TiDB	5	4	0	1
Total	**44**	**42**	**9**	**2**

even invited us to help them find more. For example, a developer from PolarDB commented: “Outstanding work, this tool looks very useful, and we will fully support any technical requirements you may need." Additionally, we received positive feedback on public bug tracking platforms. Among the confirmed bugs in MariaDB, all 7 were labeled as “Major," representing the highest severity level, while all 10 bugs reported to Percona were marked as “Medium," indicating their significant impact.

Throughput. We evaluate the efficiency of test generation by measuring the number of test cases produced per second, where each test case consists of an original query and its corresponding property-equivalent mutated query. FPMT achieves a generation throughput comparable to existing DBMS testing tools [2, 7, 26]. Specifically, FPMT generates 1,000 test cases/sec for MySQL, 990 for MariaDB, 998 for OceanBase, 990 for Percona, 998 for PolarDB, and 1,000 for TiDB. Beyond competitive throughput, FPMT attains high execution success rates on most DBMSs: 89% for MySQL, 87% for MariaDB, 84% for OceanBase, 87% for Percona, 80% for PolarDB, and 86% for TiDB. These results indicate that embedding complex built-in SQL function expressions into rich-context queries does not introduce excessive syntax/semantic errors, and FPMT can efficiently generate a large volume of executable tests for effective logical bug detection.

5.3 Bug Diversity

For the 42 confirmed logical bugs, we investigate their diversity from the aspect of the function types whose evaluations trigger the inconsistencies. Table 4 summarizes the function types and the corresponding mutation rules that trigger the confirmed bugs. Overall, the confirmed bugs span five function categories, indicating that FPMT is not limited to a specific function family. Specifically, 19 confirmed bugs are associated with aggregate functions, 11 with mathematical functions, 7 with date/time functions, 4 with conversion functions, and 1 with string functions. This diversity stems from FPMT’s function-centric design: it explicitly models each target function’s signatures, constraints, and semantic properties, enabling FPMT to generate admissible, type-correct function calls and embed them into rich-context queries across different operators. Moreover, the oracle is defined

Table 4. Function types and triggering mutation rules for confirmed bugs

ID	Bug_id	Type	Rule
MySQL			
1	MySQL#119724	Mathematical	ASIN(x)→ASIN(SIN(ASIN(x)))
2	MySQL#119714	Mathematical	ASIN(x)→-ASIN(-x)
3	MySQL#119680	Aggregate	AVG(x)→-AVG(-x)
4	MySQL#119681	Aggregate	AVG(x)→-AVG(-x)
5	MySQL#119701	Mathematical	FLOOR(x)→FLOOR(FLOOR(x))
6	MySQL#119752	Date/Time	MINUTE(x)→MINUTE(x+60 MINUTE)
7	MySQL#119056	Aggregate	SUM(x)→COUNT(x)·AVG(x)
8	MySQL#119679	Aggregate	SUM(x)→-SUM(-x)
MariaDB			
9	MDEV-#38750	Conversion	CAST(x)→CAST(CAST(x))
10	MDEV-#37729	Aggregate	MIN(x)→-MAX(-x)
11	MDEV-#38651	Date/Time	MONTH(x)→MONTH(x+12 MONTH)
12	MDEV-#37704	Aggregate	SUM(x)→COUNT(x)·AVG(x)
13	MDEV-#38588	Date/Time	YEAR(x)→YEAR(x+1 YEAR)-1
14	MDEV-#38737	Mathematical	ABS(x)→ABS(-x)
15	MDEV-#38749	String	RTRIM(x)→RTRIM(RTRIM(x))
OceanBase			
16	OceanBase#2366	Date/Time	DAY(x)→DAY(x+365 DAY)
17	OceanBase#2321	Aggregate	SUM(x)→COUNT(x)·AVG(x)
18	OceanBase#2369	Aggregate	VARIANCE(x)→VARIANCE(-x)
19	OceanBase#2370	Mathematical	COS(x)→COS(ACOS(COS(x)))
Percona			
20	Percona#PS-10441	Mathematical	ASIN(x)→-ASIN(-x)
21	Percona#PS-10279	Aggregate	AVG(x)→-AVG(-x)
22	Percona#PS-10439	Mathematical	FLOOR(x)→FLOOR(FLOOR(x))
23	Percona#PS-10272	Aggregate	MIN(x)→-MAX(-x)
24	Percona#PS-10449	Aggregate	SUM(x)→-SUM(-x)
25	Percona#PS-10480	Date/Time	MINUTE(x)→MINUTE(x+60 MINUTE)
26	Percona#PS-10507	Aggregate	SUM(x)→COUNT(x)·AVG(x)
27	Percona#PS-10508	Aggregate	SUM(x)→-SUM(-x)
28	Percona#PS-10509	Aggregate	AVG(x)→-AVG(-x)
29	Percona#PS-10510	Mathematical	ASIN(x)→ASIN(SIN(ASIN(x)))
PolarDB			
30	PolarDB#244	Mathematical	SIN(x)→SIN(x+2·PI())
31	PolarDB#245	Aggregate	AVG(x)→-AVG(-x)
32	PolarDB#257	Aggregate	AVG(x)→-AVG(-x)
33	PolarDB#259	Aggregate	AVG(x)→-AVG(-x)
34	PolarDB#260	Date/Time	DAY(x)→DAY(x+365 DAY)
35	PolarDB#258	Date/Time	YEAR(x)→YEAR(x+1 YEAR)-1
36	PolarDB#261	Aggregate	SUM(x)→-SUM(-x)
37	PolarDB#262	Mathematical	FLOOR(x)→FLOOR(FLOOR(x))
38	PolarDB#263	Mathematical	MAX(x)→-MIN(-x)
TiDB			
39	TiDB#63643	Aggregate	MAX(x)→-MIN(-x)
40	TiDB#66322	Conversion	NULLIF(x)→NULLIF(NULLIF(x))
41	TiDB#66323	Conversion	NULLIF(x)→NULLIF(NULLIF(x))
42	TiDB#66324	Conversion	CAST(x)→CAST(CAST(x))

at the *function-expression* level. FPMT rewrites only the target function into a property-equivalent expression, leaving the surrounding query context unchanged. This approach ensures that the same testing principle applies consistently across different function types and can detect inconsistencies when the DBMS violates the modeled function property during evaluation.

Case Study. Figure 5 shows a bug found by FPMT when transforming `AVG(x)` to `-AVG(-x)` (PolarDB#259 [28]). We observed an inconsistency in the results. Figure 6 shows that the root cause of this bug lies in improper type inference and type merging. First, the result of the minus operation (-) is incorrectly inferred as decimal(64, 0), which leads to a mismatch in the expected precision. When merging decimal

```
CREATE TABLE t1 (c1 INT);
INSERT INTO t1 (c1) VALUES (4574);

-- Original Query
WITH cte_619 AS ( SELECT (SELECT AVG(48) AS subq_col FROM t1 AS s890) AS
col_1, (SELECT SUM(1) AS subq_col FROM t1 AS s180) AS col_2, AVG(anl57.c1)
AS col_3 FROM t1 AS anl57)  SELECT riq85.col_3 AS col_1, MINUTE('2023-01-01')
AS col_2 FROM cte_619 AS riq85 EXCEPT SELECT DISTINCT wao91.col_1 AS
col_1, wao91.col_3 AS col_2  FROM cte_619 AS wao91;
--result:    4574.00000000000000000000 |  20.00000000000

-- Mutated Query
WITH cte_619 AS (SELECT (SELECT -AVG(-48) AS subq_col FROM t1 AS s890)
AS col_1, (SELECT SUM(1) AS subq_col FROM t1 AS s180) AS col_2,
AVG(anl57.c1) AS col_3 FROM t1 AS anl57) SELECT riq85.col_3 AS col_1,
MINUTE('2023-01-01') AS col_2 FROM cte_619 AS riq85 EXCEPT SELECT
DISTINCT wao91.col_1 AS col_1, wao91.col_3 AS col_2 FROM cte_619 AS wao91;
--result:    4574.00000000000000000000 |  20.00000000000    --(expected)
--result:    4574 |  20.00000000000    --(actual)
```

Figure 5. Case study: query result inconsistency in PolarDB.

```
// Incorrect Type Inference: The dout variable is incorrectly inferred as decimal(64,
0) due to the improper handling of the minus operation.
int dout = Math.max(p1 - s1, p2 - s2);
dout = Math.min(dout, maxPrecision);

// Type Merging Issue: In PolarDB, the type merging process gives priority to the
integer part of the decimal type, while MySQL 8.0 prioritizes the fractional part
(scale).
int scale = Math.max(s1, s2);
scale = Math.min(scale, maxPrecision - dout);
scale = Math.min(scale, maxScale);

// Precision Loss: The incorrect type inference and merging process leads to the
final precision calculation (dout + scale), resulting in lost precision.
int precision = dout + scale;
assert precision <= maxPrecision;
assert precision > 0
   || (resultType.getSqlTypeName() == SqlTypeName.DECIMAL && precision == 0
&& scale == 0);

resultType = createSqlType(SqlTypeName.DECIMAL, precision, scale);
```

Figure 6. Root cause of decimal type inference and merging bug in PolarDB.

types, PolarDB and MySQL 8.0 handle the priority between precision and scale differently, with PolarDB giving precedence to the integer part and MySQL 8.0 prioritizing the fractional part. This difference in handling results in an incorrect type being inferred. Finally, the optimizer's CAST function, which uses this erroneous type, leads to precision loss during query execution.

5.4 Comparative Study

To demonstrate that FPMT detects logical bugs in built-in SQL functions missed by existing methods, we compare FPMT with state-of-the-art DBMS testing approaches from two perspectives: testing tools and timing of bug detection.

Tool Comparison. At the tool level, we selected six state-of-the-art logical bug detection approaches for comparison: PQS [31], NoREC [29], TLP [30], DQE [32], Pinolo [11] and EET [14]. We run each tool on the latest version of each supported DBMS, with a testing duration of 24 hours. Since the baseline methods do not support all DBMSs tested by FPMT our comparison is limited to the DBMSs commonly supported by FPMT and each baseline method. Table 5 shows that FPMT outperforms the other tools in detecting logical bugs across all DBMSs, especially in MySQL. Specifically, FPMT found 13,7,7,20,22 and 10 more bugs than PQS, NoREC, TLP, DQE, Pinolo and EET. Note that the increments are calculated based only on the DBMSs commonly supported by both approaches. The other approaches were less effective in our experiments because we tested the latest versions of the DBMSs. We further investigated the relation between the bugs detected by other tools and FPMT, to check if FPMT can find bugs missed by existing tools. We found that the 44 logical bugs detected by FPMT are entirely disjoint from the 12 logical bugs detected by other tools, indicating that FPMT is capable of discovering bugs missed by the other tools, and that it complements the existing methods. Notably, all the bugs detected by FPMT are related to the implementation of built-in SQL functions, further highlighting its ability to pinpoint issues specific to function behavior in DBMSs.

Table 5. Number of logical bugs triggered in 24 hours.

DBMS	PQS	NoREC	TLP	DQE	Pinolo	EET	FPMT
MySQL	0	-	4	0	2	3	8
MariaDB	-	0	-	0	1	-	7
OceanBase	0	-	-	-	0	-	5
Percona	-	-	-	-	-	-	10
PolarDB	-	-	-	-	-	-	9
TiDB	-	-	2	0	0	0	5
Total	0	0	6	0	3	3	44
Increment	13	7	7	20	22	10	

Note."-" indicates that the tool does not support the DBMS. "Increment" is computed on the commonly supported DBMSs between the baseline tool and FPMT.

Temporal Comparison. We also present a comparative study from a temporal perspective following EET [14], aiming to evaluate whether FPMT can uncover logical bugs missed by prior approaches. We analyze the earliest bug-involved version of each of the 42 confirmed logical bugs discovered by FPMT, and examine whether these versions predate the publication of existing approaches [4, 6, 11, 14, 29–34]. If FPMT discovers long-standing bugs that have persisted for years without being reported by developers or uncovered by other methods, we conclude that FPMT is capable of identifying logical bugs overlooked by prior techniques. This comparison is both reasonable and objective for two main reasons: (1) None of the logical bugs found by FPMT were marked as duplicates by the respective DBMS vendors, indicating that these bugs remained undetected by other methods prior to our discovery; (2) The majority of the evaluated DBMSs have already been extensively tested by existing approaches, yet they still failed to reveal the long-lived logical bugs identified by FPMT during their comprehensive evaluation campaigns.

As shown in Table 6, among the 42 confirmed bugs, 20 bugs originated before 2020, indicating that all post-2020 approaches (e.g., PQS [31], TLP [30], NoREC [29]) failed to

detect these long-standing issues in their extensive evaluations. In addition, 21 bugs were already present between 2020 and 2023, yet were not identified by testing approaches introduced after 2023, such as EET [14], Pinolo [11] and DQE [32]. These findings show that existing approaches indeed missed a substantial number of long-lived logical bugs. This is because FPMT focuses on *function property-driven metamorphic testing*, concentrating on the semantic properties of SQL functions to uncover logical bugs within the function's implementation. However, other approaches focus on validating query semantics at the relational-operator and plan/optimization level, aiming to uncover planner/optimizer issues. Furthermore, we investigate the longest latency of the confirmed bugs for each DBMS. As reported in Table 6, FPMT identifies at least one bug with a latency exceeding two years in every tested DBMS, with the longest latency reaching **20 years**. These results demonstrate that FPMT is highly effective at discovering long-standing, previously unknown logical bugs across diverse DBMSs.

Table 6. Latency of the confirmed logical bugs

DBMS	Found	Bug-involved year		Longest latency
		< 2023	< 2020	
MySQL	8	8	6	20 years
MariaDB	7	6	4	13 years
OceanBase	4	4	0	3 years
Percona	10	10	10	12 years
PolarDB	9	9	0	2 years
TiDB	4	4	0	2 years
Total	**42**	**41**	**20**	**20 years**

6 Related Work

Logical Bug Detection in DBMSs. A variety of approaches have been proposed to detect logical bugs in DBMSs [4, 6, 11, 13, 14, 29–34]. For example, NoREC [29] transforms an optimizable query into a non-optimizable form and detects semantic inconsistencies by comparing their outputs. TLP [30] divides query predicates into multiple subqueries and verifies that the union of their results is equivalent to the original query. PQS [31] constructs queries expected to retrieve a specific pivot row, while DQE [32] detects logical bugs by comparing whether different SQL queries with the same predicate access the same rows in the database. Pinolo [11] generalizes this paradigm through set-level approximation, where predicates are relaxed or restricted to generate over- and under-approximate queries, and correctness is judged by inclusion or containment relations between result sets. EET [14] applies expression-preserving transformations, and Radar [33] compares query results between databases with and without metadata to identify semantic flaws.

As discussed in Section 1 and Section 2, FPMT is orthogonal to these prior efforts. Most existing techniques design oracles around query structure and relational operators , and thus excel at exposing bugs in query planning and optimization. However, they are less effective for logical bugs rooted in built-in SQL functions. In contrast, FPMT targets data-operation-level logical bugs inside built-in SQL function implementations by constructing MRs directly from function properties, enabling function-specific mutations and oracles that remain applicable even when the function is embedded in complex query contexts.

DBMS Test-Case Generation. A variety of approaches have been proposed for generating diverse test cases for DBMSs, with the aim of improving coverage and revealing potential bugs. These techniques [2, 9, 12, 17, 40] typically focus on generating valid and varied SQL queries, but do not specifically target logical bugs. For example, SQLsmith [2] is a grammar-based DBMS fuzzer that embeds SQL grammar rules to generate complex SQL queries. It uses a random walk approach to explore the SQL syntax and generate a wide range of queries. SQUIRREL [40] is a mutation-based DBMS fuzzer which introduces an intermediate representation for SQL queries and models dependencies between SQL statements, enabling the generation of queries that contain multiple SQL operations. Griffin [9] uses a grammar-free mutation approach, where it mutates SQL queries based on DBMS state information encapsulated in a metadata graph. DynSQL [12] takes a dynamic approach by interacting with the DBMS to capture the latest state information, allowing for the incremental generation of valid and complex queries. These techniques aim to prevent semantic errors and improve the diversity of generated queries. In addition to general query generation, some approaches have been specifically designed to aid in bug detection. SQLRight [15] leverages code coverage feedback to enhance test-case generation. QPG [3] takes a different approach by recording the query plans covered during DBMS testing and prioritizing the mutation of queries that trigger new query plans.

These query generation approaches complement FPMT. They are effective at producing diverse, context-rich queries, while FPMT can serve as a lightweight add-on that embeds complex built-in function expressions into these queries and applies function property-driven metamorphic relations as test oracles to detect logical bugs in built-in SQL functions.

7 Conclusion

In this paper, we introduce *Function Property-Driven Metamorphic Testing* to systematically detect logical bugs in built-in SQL functions. By focusing on the semantic properties of SQL functions, our method enables the generation of property-equivalent queries, effectively exposing bugs that are difficult to detect with existing testing techniques. The evaluation results across multiple DBMSs demonstrate FPMT's ability to uncover previously unknown logical bugs, highlighting its potential to enhance the reliability of SQL function implementations.

References

[1] Anonymous. 2023. A Detailed Example of Embedding in Rich-context Query (Section 3.3). https://anonymous.4open.science/r/FPMT-DBTesting-D3CA/A%20Detailed%20Example%20of%20Embedding%20in%20Rich-context%20Query%20(Section%203.3).md Accessed: 2026-03-05.

[2] Anse1. 2020.. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed Dec 2025.

[3] Jinsheng Ba and Manuel Rigger. 2023. Testing database engines via query plan guidance. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2060–2071.

[4] Jinsheng Ba and Manuel Rigger. 2024. Keep it simple: Testing databases via differential query plans. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[5] Alibaba Cloud. 2023. PolarDB Product Overview. https://help.aliyun.com/zh/polardb/product-overview/. Accessed Dec 2025.

[6] Wenqian Deng, Jie Liang, Zhiyong Wu, Jingzhou Fu, and Yu Jiang. 2025. Detecting Logic Bugs in DBMSs via Equivalent Data Construction. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[7] SQLancer developers. 2025. SQLancer: Automated testing to find logic and performance bugs in database systems. https://github.com/sqlancer/sqlancer. Accessed Dec 2025.

[8] MariaDB Foundation. 2023. MariaDB. https://mariadb.org/ Accessed Dec 2025.

[9] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang, and Yu Jiang. 2022. Griffin: Grammar-free dbms fuzzing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE 22)*. 1–12.

[10] Jingzhou Fu, Jie Liang, Zhiyong Wu, Yanyang Zhao, Shanshan Li, and Yu Jiang. 2025. Understanding and Detecting SQL Function Bugs: Using Simple Boundary Arguments to Trigger Hundreds of DBMS Bugs. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1061–1076.

[11] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting logical bugs in database management systems with approximate query synthesis. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 345–358.

[12] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. {DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4949–4965.

[13] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting transactional bugs in database engines via {graph-based} oracle construction. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 397–417.

[14] Zu-Ming Jiang and Zhendong Su. 2024. Detecting logic bugs in database engines via equivalent expression transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 821–835.

[15] Yu Liang, Song Liu, and Hong Hu. 2022. Detecting logical bugs of {DBMS} with coverage-based guidance. In *31st USENIX Security Symposium (USENIX Security 22)*. 4309–4326.

[16] Li Lin, Zongyin Hao, Chengpeng Wang, Zhuangda Wang, Rongxin Wu, and Gang Fan. 2024. SQLess: Dialect-Agnostic SQL Query Simplification. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 24)*. 743–754.

[17] Li Lin, Qinglin Zhu, Hongqiao Chen, Zhuangda Wang, Rongxin Wu, and Xiaoheng Xie. 2025. QTRAN: Extending Metamorphic-Oracle Based Logical Bug Detection Techniques for Multiple-DBMS Dialect Support. In *Proceedings of the 34nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 25)*. 731–752.

[18] linli1724647576. 2026. ValScope. GitHub repository. https://github.com/linli1724647576/ValScope Accessed: 2026-02-07.

[19] Toby Mao. 2024. SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine. https://github.com/tobymao/sqlglot Accessed Dec 2025.

[20] MySQL. 2023. MySQL. https://www.mysql.com/ Accessed Dec 2025.

[21] OceanBase. 2023. OceanBase Open Source. https://www.oceanbase.com/product/opensource Accessed Dec 2025.

[22] Oracle. 2026. Chapter 6: Adding Functions to MySQL. https://dev.mysql.com/doc/extending-mysql/8.3/en/adding-functions.html. Accessed: 2026-01-23.

[23] Oracle. 2026. Functions and Operators. https://dev.mysql.com/doc/refman/8.0/en/functions.html. Accessed: 2026-01-23.

[24] Percona. 2023. Percona - Open Source Database Software and Services. https://www.percona.com/ Accessed Dec 2025.

[25] Percona. 2026. PS-10441: Queries exposing a logical bug in built-in SQL function. https://perconadev.atlassian.net/browse/PS-10441. Accessed: 2026-02-04.

[26] PingCap. 2022. go-randgen. https://github.com/pingcap/go-randgen Accessed Dec 2025.

[27] PingCAP. 2023. TiDB. https://www.pingcap.com/tidb/ Accessed Dec 2025.

[28] PolarDB. 2023. Issue 259: Bug Report for PolarDBX-SQL. https://github.com/polardb/polardbx-sql/issues/259 Accessed: 2023-12-07.

[29] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152.

[30] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.

[31] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 667–682.

[32] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing database systems via differential query execution. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2072–2084.

[33] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1884–1897.

[34] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting logic bugs of join optimizations in dbms. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.

[35] The PostgreSQL Global Development Group. 2015. Security Update Release 16.15. https://www.postgresql.org/about/news/2015-10-08-security-update-release-1615/. Accessed: 2026-02-03.

[36] The PostgreSQL Global Development Group. 2026. Chapter 9. Functions and Operators. https://www.postgresql.org/docs/current/functions.html. Accessed: 2026-01-23.

[37] The PostgreSQL Global Development Group. 2026. User-Defined Functions. https://www.postgresql.org/docs/current/xfunc.html. Accessed: 2026-01-23.

[38] Rick F. Van Der Lans. 1989. *The SQL Standard: A Complete Guide Reference.* Prentice Hall International (UK) Ltd.

[39] Wikipedia. 2026. Databases. https://en.wikipedia.org/wiki/Database. Accessed: 2026-01-23.

[40] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 955–970.